#7 We throw an error that can be handled by the client.
#8 Compute the optimal number of hashes function needed: `k = m/n * ln(2)`.
#9 The number of elements for the (integer) buffer is computed by dividing the number of total bits needed by the number of bits per int. Notice how we use the ceiling function here
#10 Create the buffer that will store the filter's bits, all initialized to `0`.
#11 Create and store the hash functions that will be used to get the bit indices for a key.

On creation, we need to provide only the maximum number of elements that the filter will be expected to contain. If, at any time, we realize that we stored in the filter more than `maxSize` elements, the good news is that we wouldn't run out of space, but the bad news is that we could not guarantee the expected precision anymore.

Speaking of which, we can pass an optional second parameter to set the expected accuracy: by default, the threshold for the probability of a false positive (`maxTolerance`) is set to 1%, but we can decide to aim for better accuracy by passing a smaller value, or to settle for worse accuracy, as a tradeoff for a lower amount of memory needed, by passing a higher value.

The last optional parameter is needed, as explained in the previous section, to force a deterministic behavior for the filter. When omitted by the caller, a random value is generated for the seed.